Volume 4: Store, Cosmetics Catalog & Virtual Economy

Technical specification for store item catalogs, virtual pricing models, atomic purchase verification, inventory ownership records, and cosmetic texture rendering.

Target Stack: NestJS 11 · Prisma 6 · PostgreSQL · Next.js · **Lead:** Muhammad Sameer Ali

VOLUME OBJECTIVE:
Specifies the virtual store, catalog pricing in USDT and loyalty points, item unlock validation, double-entry payment deduction, and inventory persistence across frontend card previews, 3D cylinder shaders, and multiplayer battlecards.

1. Store Catalog Architecture & Items Registry

• **Catalog Endpoint:** GET /shop returns categorized list of available cosmetics with price, rarity badge (Common, Rare, Epic, Legendary), preview assets, and discount tags.
• **Item Categories:** Card Skins, Avatar Accessories (Hats, Eyewear), Outfits, Shoes, Board Skins, Victory Mascot Dances, Sound FX Packs.
• **Free Defaults:** Basic assets are free for all registered and guest players and do not require purchase.

2. Atomic Purchase Flow & Inventory Synchronization

Step / Phase	Action & Payload	Validation & Database Updates
1. Selection	Player clicks Buy in app/shop/page.tsx.	Client checks balance against useWallet().
2. Request	POST /shop/purchase { itemId: string, currency: 'USDT'|'COINS' }	Backend verifies item is active and user doesn't already own it.
3. Deduction	Double-entry ledger entry appended.	LedgerEntry(amount: -price, refType: 'SHOP_PURCHASE').
4. Unlock	Item ID appended to User.cosmeticsOwned.	User's owned inventory array updated atomically.
5. Equip	PATCH /me/cosmetics { slot: 'head', itemId: 'crown-gold' }	Verifies ownership in cosmeticsOwned, updates User.cosmeticsJson.

3. Cosmetic Rendering & Battle Integration

Equipped cosmetics are broadcasted in match:state_sync so all opponents see custom 3D card backs, avatar skins, and particle trails on the desktop 3-column arena and mobile battle strip.